

Clasificación del Fashion-MNIST dataset con una Red Neuronal Feedforward en PyTorch

Juliana Canto* and Diego Jesús Scacchi**

Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina
(Dated: 9 de noviembre de 2025)

En este trabajo se construye una red neuronal *feedforward* para clasificar imágenes a partir de la base de datos de Fashion-MNIST [1], usando la librería PyTorch [2]. La metodología empleada incluye preprocesamiento, variación en el diseño de arquitectura de la red y modificaciones en los hiperparámetros que la constituyen. Los resultados muestran que la red logra identificar prendas de ropa con precisión y sin sobreajuste, y que la elección de hiperparámetros es clave a la hora de mejorar su rendimiento.

I. INTRODUCCIÓN

Las redes neuronales artificiales son un modelo de machine learning que pretende tomar decisiones imitando la manera en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones. Estos modelos computacionales permiten resolver problemas complejos de clasificación y regresión estableciendo conexiones entre las neuronas artificiales. El creciente avance de la inteligencia artificial y los modelos de aprendizaje autónomo permiten el diagnóstico médico mediante la clasificación de imágenes, predicciones financieras mediante el procesamiento de datos históricos de instrumentos financieros, identificación de compuestos químicos y muchas otras utilidades.

En este trabajo se utilizó la base de datos de Fashion-MNIST [1], la cual es un conjunto de imágenes de 28×28 píxeles que representan 10 categorías de prendas de vestir desde remeras a sandalias y buzos entre otras. Esta base de datos permite evaluar y entrenar modelos de redes neuronales en tareas clasificación de información. En conjunto con la librería PyTorch [2] es posible diseñar, entrenar y validar una red neuronal multicapa para clasificar estas imágenes. Nuestro objetivo se basará en poder clasificarlas correctamente, variando los distintos hiperparámetros propios de la red.

II. TEORÍA

Las redes neuronales artificiales están compuestas por capas de unidades interconectadas, denominadas neuronas, organizadas en tres estructuras principales: una capa de entrada, capas ocultas y una capa de salida. Estas redes están diseñadas para procesar información de manera jerárquica, transformando los datos desde su estado inicial hasta una salida que representa el resultado del modelo.

En particular, las redes neuronales de tipo *feedforward* se caracterizan porque la información fluye únicamente en una dirección: desde la capa de entrada hacia la de sa-

lida, pasando secuencialmente por las capas ocultas. No existen conexiones que retroalimenten las salidas hacia capas anteriores, lo que las distingue de las redes recurrentes. Cada neurona recibe señales de la capa previa, realiza una combinación lineal de las entradas y aplica una función de activación para generar su salida, que servirá como entrada para la siguiente capa.

El flujo de datos comienza en la capa de entrada, que recibe las características del problema sin procesar. En este trabajo, la información proviene del conjunto de datos [1].

En las capas ocultas, cada neurona combina las entradas ponderadas por sus respectivos pesos (w_{ij}) junto con un sesgo (b). El sesgo actúa como un ajuste adicional que permite desplazar la activación de la neurona, otorgando mayor flexibilidad al modelo. El cálculo lineal realizado por cada neurona puede expresarse como:

$$z = w \cdot x + b \quad (1)$$

donde $a = f(z)$ representa la activación resultante. Para introducir no linealidad en el modelo, se utiliza una función de activación f . Existen diversas opciones, como la sigmoide o la tangente hiperbólica, pero en este trabajo se emplea la *Rectified Linear Unit* (ReLU), definida como:

$$f(z) = \max(0, z) \quad (2)$$

La función ReLU introduce no linealidad de manera sencilla y resulta computacionalmente eficiente, ya que solo requiere comparar el valor de entrada con cero. Además, su derivada simple contribuye a mitigar el problema del *vanishing gradient* en redes profundas.

La capa de salida de la red transforma los valores lineales obtenidos en scores mediante la función *softmax*, permitiendo interpretar cada salida como la probabilidad de pertenencia a una clase específica. El aprendizaje del modelo se guía mediante una **función de pérdida** que cuantifica la discrepancia entre las predicciones $\hat{y}$ y las

etiquetas reales y . Para problemas de clasificación multi-clase, se utiliza la *entropía cruzada* (*Cross-Entropy Loss*), expresada como:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j}) \quad (3)$$

donde N es el número total de ejemplos, C el número de clases, $y_{i,j}$ indica si el ejemplo i pertenece a la clase j , y $\hat{y}_{i,j}$ es la probabilidad predicha para esa clase. Esta función mide el error de clasificación y orienta el proceso de ajuste de los parámetros.

La optimización de la red consiste en actualizar los pesos ω para minimizar la función de pérdida. Esto se realiza mediante el algoritmo de *retropropagación*, que aplica el método de *descenso por gradiente*. La regla de actualización de los pesos se expresa como:

$$\omega \leftarrow \omega - \eta \frac{\partial L}{\partial \omega} \quad (4)$$

donde η es la tasa de aprendizaje, un hiperparámetro que controla la magnitud de los ajustes. En este trabajo se utilizó el optimizador *Stochastic Gradient Descent* (SGD), que actualiza los pesos en base a pequeños lotes (*mini-batches*) de datos, permitiendo un entrenamiento más eficiente y una convergencia estable sin necesidad de ajustar dinámicamente la tasa de aprendizaje. Posteriormente, se considera el cambio al algoritmo adaptativo *Adam*.

Asimismo, es importante analizar el impacto de los distintos hiperparámetros del modelo, ya que determinan en gran medida su rendimiento y capacidad de generalización. Entre los más relevantes se encuentran la *tasa de aprendizaje* (η), que controla la magnitud de los ajustes durante la optimización; el *optimizador* empleado (por ejemplo, *SGD* o *Adam*); el *valor de dropout*, que desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento para reducir el sobreajuste; el *número de neuronas* en las capas ocultas, que define la capacidad de la red para representar relaciones no lineales; el *número de épocas de entrenamiento*, que determina cuántas veces el modelo recorre todo el conjunto de datos; y el *tamaño de los lotes* (*batch size*), que influye en la estabilidad y velocidad del aprendizaje.

El conjunto de datos a analizar contiene un total de 70 000 imágenes en escala de grises, de las cuales 60 000 se utilizan para entrenamiento y 10 000 para validación o prueba. El uso diferenciado de los conjuntos de entrenamiento y validación resulta fundamental para medir el desempeño real del modelo. Mientras que el conjunto de entrenamiento permite ajustar los parámetros internos de la red neuronal, el conjunto de validación ofrece una evaluación independiente que ayuda a detectar sobreajuste y a estimar la capacidad de generalización del modelo

frente a datos no vistos. Cada imagen representa una prenda de vestir perteneciente a una de diez categorías: remera/top, pantalón, pulóver, vestido, abrigo, sandalia, camisa, zapatilla, bolso y bota. En la Figura 1 se ve un ejemplo de una muestra aleatoria de imágenes del conjunto de datos con sus respectivas etiquetas. El objetivo del conjunto es servir como referencia para probar algoritmos de clasificación de imágenes, permitiendo evaluar modelos de aprendizaje profundo en tareas visuales simples. En este trabajo, las imágenes fueron normalizadas y transformadas en tensores antes de ser utilizadas como entradas de la red neuronal.

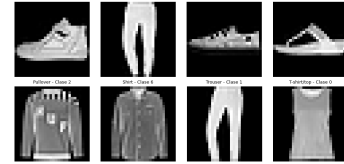


Figura 1: Ejemplo de imágenes aleatorias del conjunto de datos Fashion-MNIST con sus respectivas etiquetas.

III. RESULTADOS

Para abordar la relación entre la variación de hiperparámetros y la performance de la red neuronal, consideramos dividir el tópico en una serie de experimentos, donde se modifica solo *un* hiperparámetro por cada caso. En todos los experimentos se utilizó una arquitectura de red *feedforward* con una capa de entrada de 784 neuronas de entrada (correspondiente a los 28x28 píxeles por imagen), dos capas ocultas y una capa de salida de 10 neuronas, vinculado a la cantidad de artículos de ropa. Las funciones de activación en todos los casos serán ReLU. Las capas sucesivas están *totalmente* conectadas entre sí. Los detalles de implementación pueden encontrarse en la Tabla I. Las métricas en las que nos concentraremos para medir el desempeño de cada modelo serán la *precisión* sobre cada conjunto, y el *error promedio por lote*, esto es el valor promedio devuelto por la función de pérdida a lo largo de todos los lotes de una época.

Partimos de un modelo inicial que nos servirá como base para después analizar los comportamientos frente a las fluctuaciones de hiperparámetros.

De las curvas de entrenamiento y validación en la Figura 2 podemos observar un aprendizaje de la red progresivo y consistente, marcado por la disminución de la pérdida y el aumento de la precisión en los conjuntos *train* y *validation*. La métrica accuracy alcanza $\approx 90\%$ para el entrenamiento y un 87% para validación. Cabe destacar que hay una leve separación entre las curvas de los conjuntos aplicados, por lo que se podría decir que hay cierta tendencia al *overfitting* (no lo sería en principio aquí ya

Nº de experimento	n_1	n_2	p	Optimizador	Batch-size	Learning Rate	Epochs	Objetivo
1	128	64	0.2	SGD	100	10^{-3}	40	Modelo base
2	128	64	0.2	SGD	100	10^{-4}	40	Variar el learning rate
3	128	64	0.2	Adam	100	10^{-4}	40	Cambiar el optimizador
4	128	64	0.8	SGD	100	10^{-4}	40	Variar el dropout
5	256	128	0.2	SGD	100	10^{-4}	40	Variar n_1 y n_2
6	128	64	0.2	SGD	100	10^{-4}	100	Variar la cantidad de epochs
7	128	64	0.2	SGD	200	10^{-4}	40	Variar el <i>batchsize</i>

Tabla I: Lista de experimentos en donde se varían distintos hiperparámetros de un experimento base. En todos los casos se entrenaron los modelos partiendo de pesos sinápticos (o parámetros) inicialmente elegidos al azar.

que la diferencia de valores es $< 3\%$).

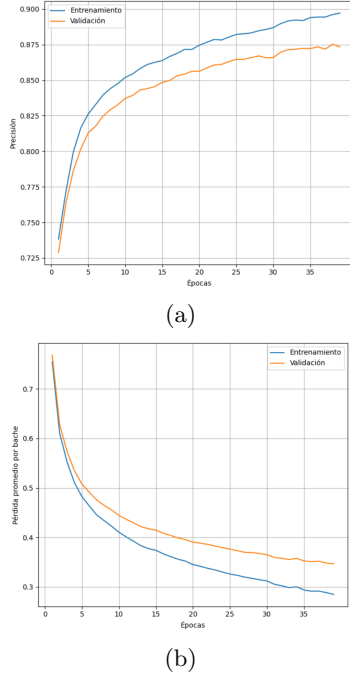


Figura 2: (a) Precisión por época de los conjuntos de entrenamiento y validación del experimento base. (b) Error promedio por lote de este último.

A. Variación de hiperparámetros: Tasa de Aprendizaje

En esta primera variante (Figura 3) modificamos el valor de lr (ó η) a 10^{-4} . Se puede ver que la red logra aprender con mayor capacidad de generalización (curvas de *train* y *val* más cercanas entre sí) y estabilidad respecto del baseline, pero a costa de una tasa de aprendizaje más lenta y menor precisión para la misma cantidad de épocas. Se realizaron otras pruebas para valores de lr más altos, como 10^{-2} , y se observó un aprendizaje más pronunciado con propensión al sobreajuste.

A partir de aquí utilizaremos $lr = 10^{-4}$ para el resto de los experimentos.

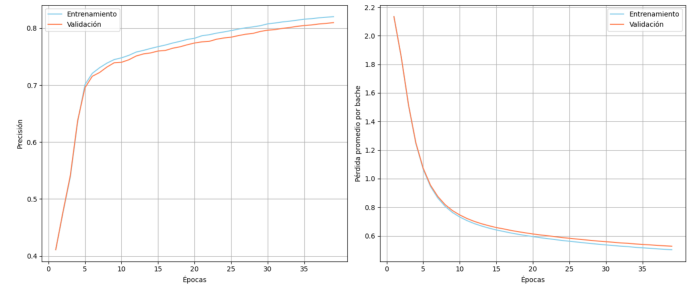


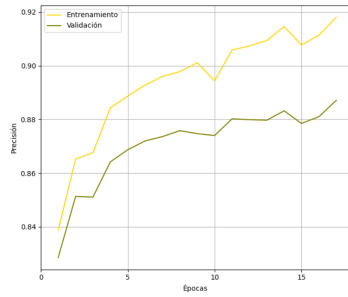
Figura 3: Accuracy y pérdida por época de los conjuntos *train/val* para el experimento 2, con $lr = 10^{-4}$.

B. Optimizador

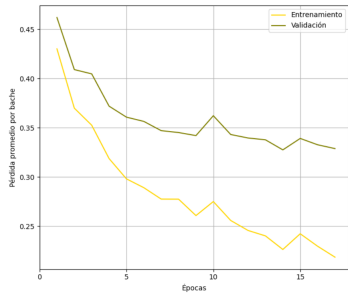
Para este experimento modificamos el optimizador con el fin de que utilice el algoritmo *Adam*. Tal como se muestra en la Figura 4, el aprendizaje de la red es muy rápido y se alcanzan precisiones (tanto en *train* como en *validation*) mayores a las obtenidas con *SGD*. El proceso de aprendizaje se ve un tanto inestable, con una brecha considerable entre las curvas. Esto último sugiere un riesgo a *overfitting*.

C. Dropout

En esta experiencia, se incrementó el valor de dropout a $p = 0.8$. Es claro en la Figura 5 que hay leves irregularidades durante el proceso de aprendizaje respecto del baseline, la precisión alcanzada se ve reducida en un $\approx 20\%$ y la pérdida es considerable incluso al finalizar las 40 épocas. Para valores de p cercanos a 0, se nota un incremento en la precisión a expensas de exhibir sobreajuste.



(a)



(b)

Figura 4: (a) Precisión por época de los conjuntos de entrenamiento y validación del experimento 3. (b) Error promedio por lote de este último.

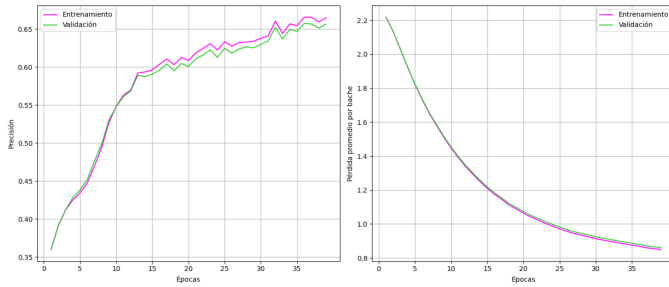


Figura 5: Precisión y pérdida por época de los conjuntos *train/val* para el experimento 4, con $p = 0.8$.

D. Número de neuronas en las capas ocultas

El cambio a una arquitectura con $n_1 = 256$, $n_2 = 128$ (ver Figura 6) en las *hidden layers* no mostró mejoras significativas respecto al modelo base. Un análisis cualitativo realizado sobre el conjunto de validación para distintos números de neuronas, que se encuentra en la Figura 7, confirma lo dicho anteriormente. Además, se observa un peor rendimiento para la configuración 64-32 y uno mejor para la configuración 512-256.

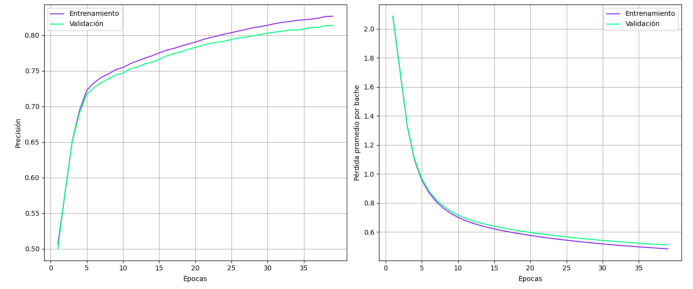


Figura 6: Precisión y pérdida por época de los conjuntos *train/val* para el experimento 5, con $n_1 = 256$, $n_2 = 128$.

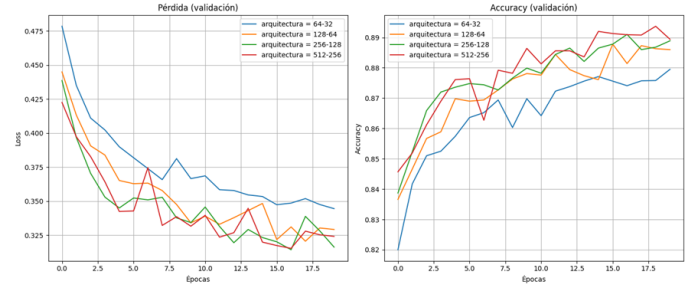


Figura 7: Pérdida y precisión sobre el conjunto de validación, iterando sobre distintos tipos de arquitectura. Efectuado para un total de 20 épocas.

E. Número de *epochs*

Para esta modificación se aumentó el número de épocas a 100. De la Figura 8 se puede deducir que a partir de 50 *epochs* el aprendizaje alcanza cierto *plateau* tanto para entrenamiento como validación, obteniendo mejoras marginales de las métricas en análisis. Un estudio en profundidad sobre el conjunto de validación que varía la cantidad de épocas logra dilucidar este fenómeno (Figura 9).

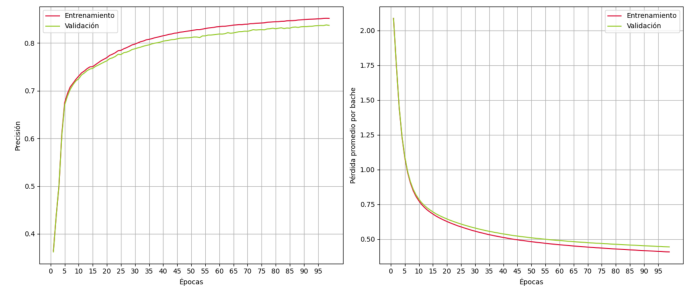


Figura 8: Precisión y pérdida de los conjuntos *train/val* para el experimento 6, donde se utilizaron 100 épocas.

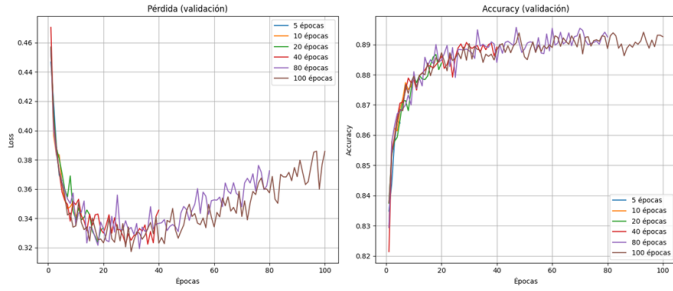


Figura 9: Pérdida y precisión sobre el conjunto de validación, iterando sobre la cantidad de épocas.

F. Batch-size

En esta simulación se fijó que la red neuronal aprenda de a lotes de 200 imágenes en vez de 100. La misma se encuentra en la Figura 10. A pesar que las fluctuaciones son sutiles, podemos ver que las curvas de *train* y *val* poseen una menor brecha entre sí, garantizando generalidad. También, el aprendizaje es más regular o *suave* comparado al baseline, a costas de obtener una menor accuracy y una mayor pérdida promedio. Experimentos adicionales con *batch-size*= 50 mostraron un mejor desempeño respecto a las métricas, pero mayor separación de las curvas.

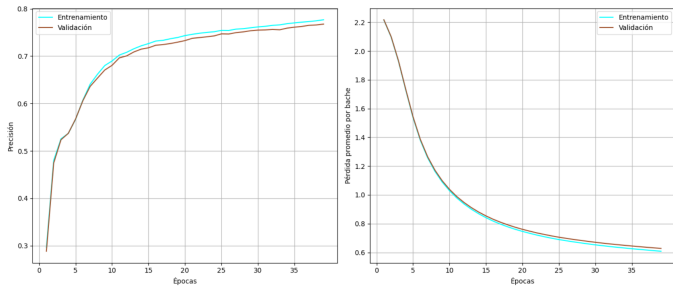


Figura 10: Precisión y pérdida por época de los conjuntos *train/val* para el experimento 7.

IV. DISCUSIÓN

Los gráficos resultantes muestran que la red *feedforward* multicapa logra clasificar precisamente las imágenes provistas [1], con una capacidad de generalización ra-

zonable en todos los experimentos analizados (variación de métricas entre *training* y *validation* < 5%). Además, evidencian cómo cada hiperparámetro impacta en el rendimiento de manera particular. Pudimos ver que la tasa de aprendizaje afecta directamente a la velocidad y estabilidad con la que aprende la red, en donde la elección de valores elevados pueden desembocar en sobreajuste. La utilización de algoritmos adaptativos como *Adam* permiten un aprendizaje más pronunciado, pero de manera inestable. Puede que sea necesario efectuar futuros experimentos donde el resto de los hiperparámetros se personalice al optimizador en cuestión para deducir resultados más contundentes. En lo que respecta al dropout p , se ve un claro vínculo con la capacidad de generalización subyacente de la red. $p = 0.8$ exhibe una pérdida de información (fue el experimento que obtuvo la menor precisión en ambos conjuntos), pero la omisión de p implicaría que la red aprenda patrones específicos del set de entrenamiento. Por otro lado, aumentar el número de neuronas en las capas ocultas nos permite obtener mejores métricas sin generar overfitting, pero solo en casos de incrementos considerables. El número de épocas por otra parte no exhibió ningún tipo de correlación significativa con la capacidad de la red para clasificar. Relacionado al *batch-size*, un balance adecuado puede contribuir a procesar las imágenes en una cantidad razonable sin comprometer el desempeño de la red. Finalmente, una posible mejora al modelo de red neuronal feedforward de este trabajo podría ser utilizar primeramente un *autoencoder* a modo de preprocesamiento, que establezca los pesos sinápticos iniciales en vez de elegirlos de manera aleatoria.

En conjunto, las variantes exploradas ponen de manifiesto la importancia de experimentar con diferentes configuraciones de la red neuronal en función del dataset a abordar y el objetivo que se quiere alcanzar.

* julicanto22@unc.edu.ar

** diego.scacchi@mi.unc.edu.ar

- [1] H. Xiao, K. Rasul, and R. Vollgraf, Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms, arXiv preprint arXiv:1708.07747 (2017).
- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimshe, L. Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, in *Advances in Neural Information Processing Systems (NeurIPS 2019)* (2019).